

Modelo Lógico + Normalização

Prof. Cloves Rocha

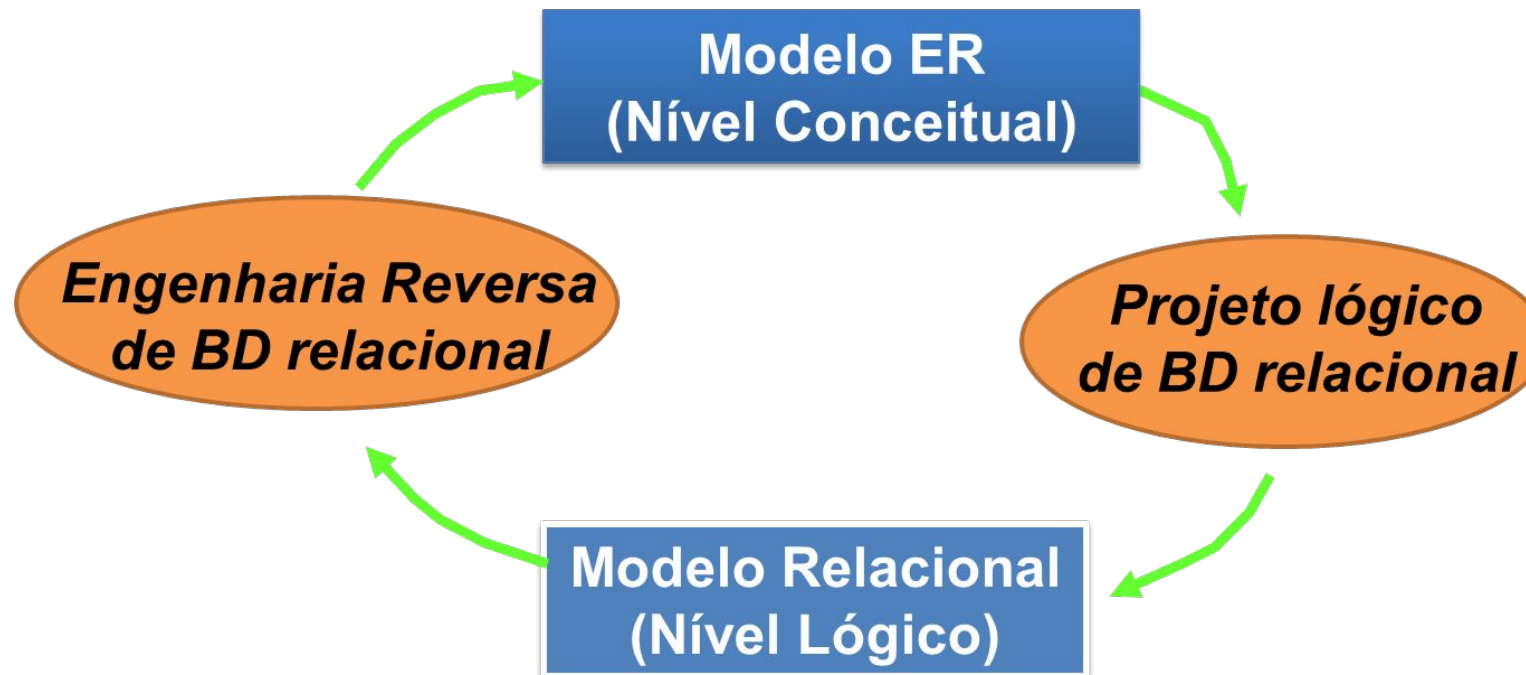
Ciência da Computação - CC

Roteiro

- Modelo Relacional
- Normalização (Formas Normais)
- Exercícios

Modelo Relacional

- Entidade □ Tabela
- Atributo □ Coluna
- Atributo Identificador □ Chave Primária



MER x Modelo Relacional

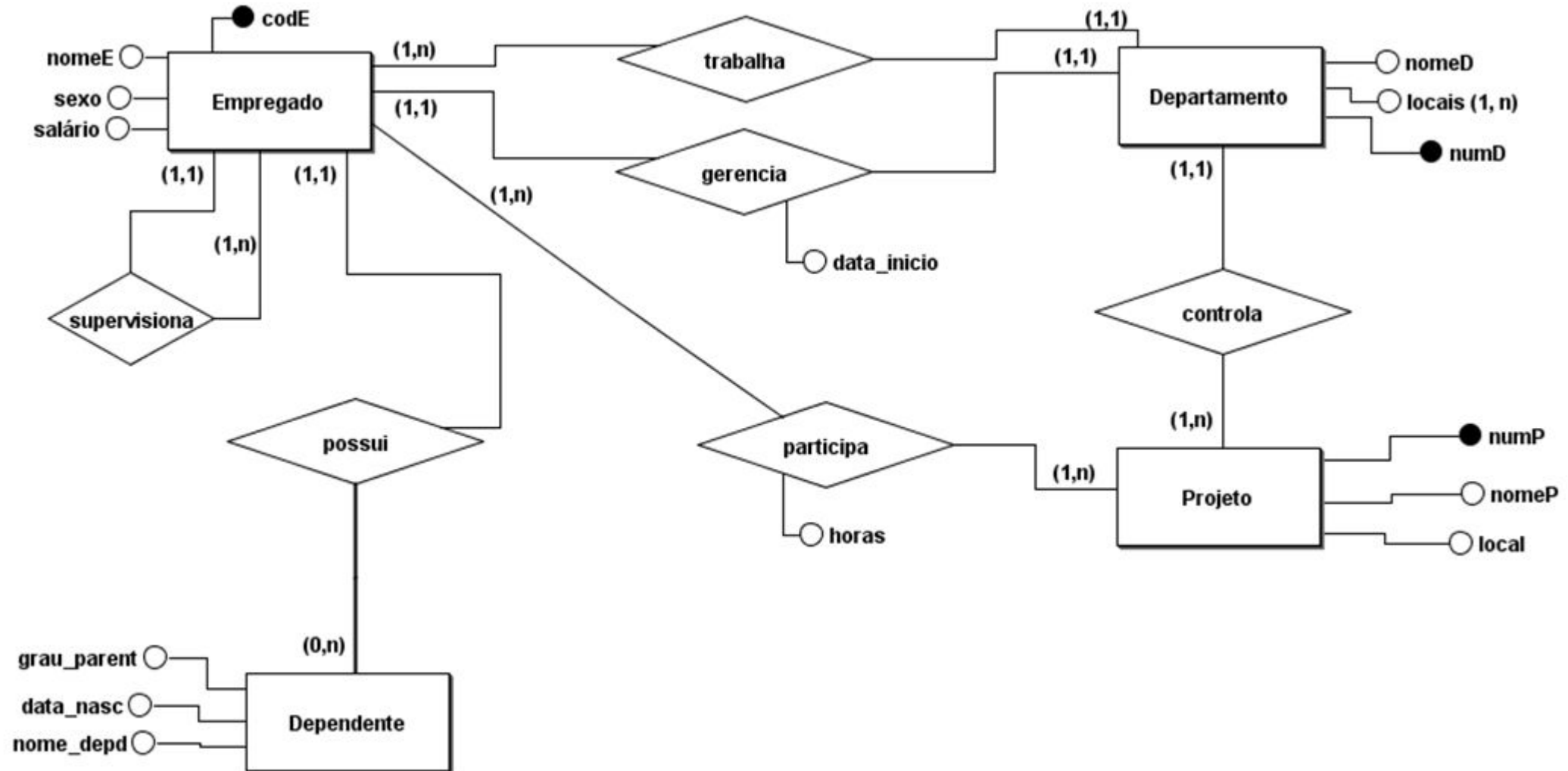
Modelo Entidade-Relacionamento

- Entidade
 - Regular
 - Fraca
- Atributo
 - Simples
 - Multivalorado
 - Composto
 - Identificador
- Relacionamento
 - 1:1, 1:n, n:m
- Generalização/Especialização

Modelo Relacional

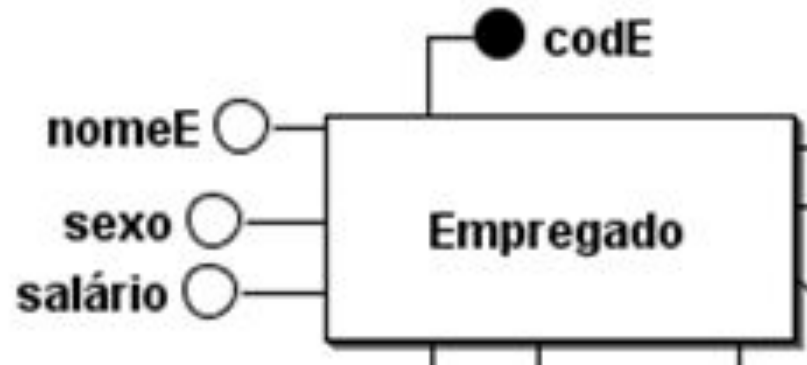
- Relação (Tabela)
- Atributo
 - Não nulos (*Not Null*), Únicos (*Unique*)
- Chave Primária (*Primary Key* - PK)
- Chave Estrangeira (*Foreign Key* - FK)
- Relacionamento
 - Usa Chave Estrangeira

DER □ Modelo Relacional



DER □ Modelo Relacional

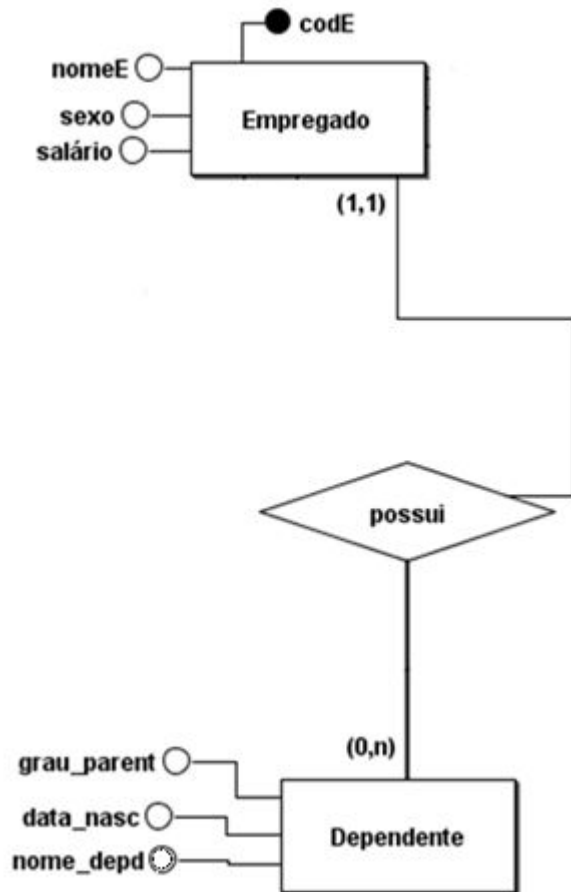
Passo 1 - Entidades Regulares



Empregado (codE, nomeE, sexo, salário)

DER □ Modelo Relacional

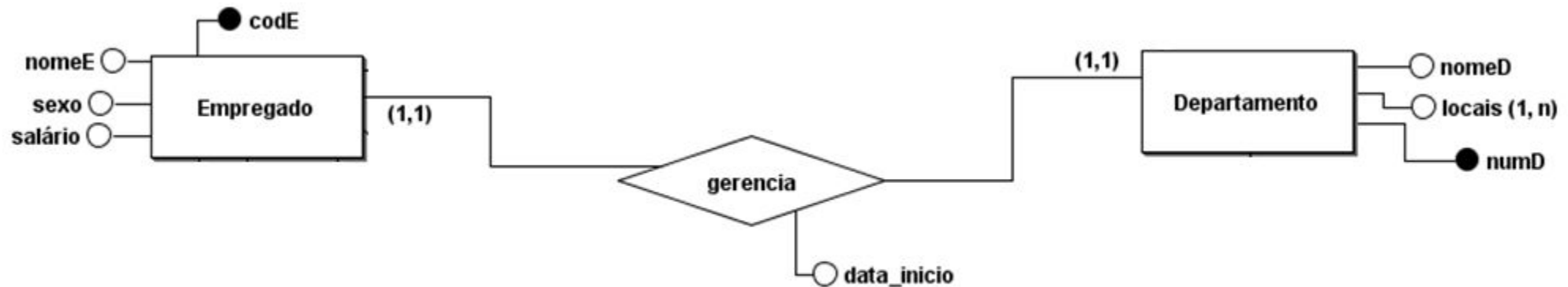
Passo 2 - Entidades Fracas (Dependentes)



Dependente (codE_{FK}, nome_depd, data_nasc, grau_parent)

DER → Modelo Relacional

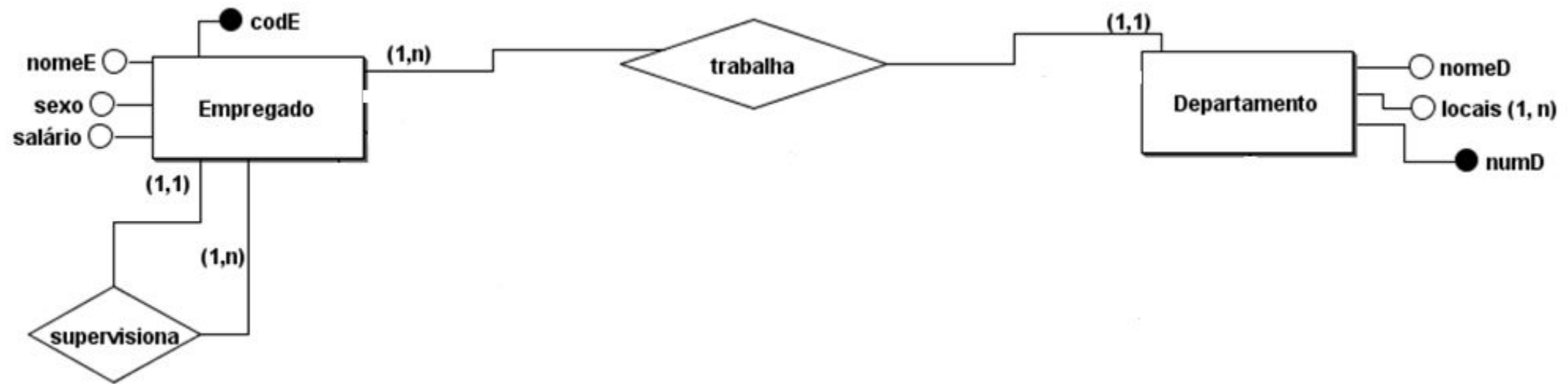
Passo 3 - Relacionamentos (1:1)



Empregado (codE, nomeE, sexo, salário, data_inicio, $numD_{FK}$) ou
Departamento (numD, nomeD, **locais**, data_início, $codE_{FK}$)

DER □ Modelo Relacional

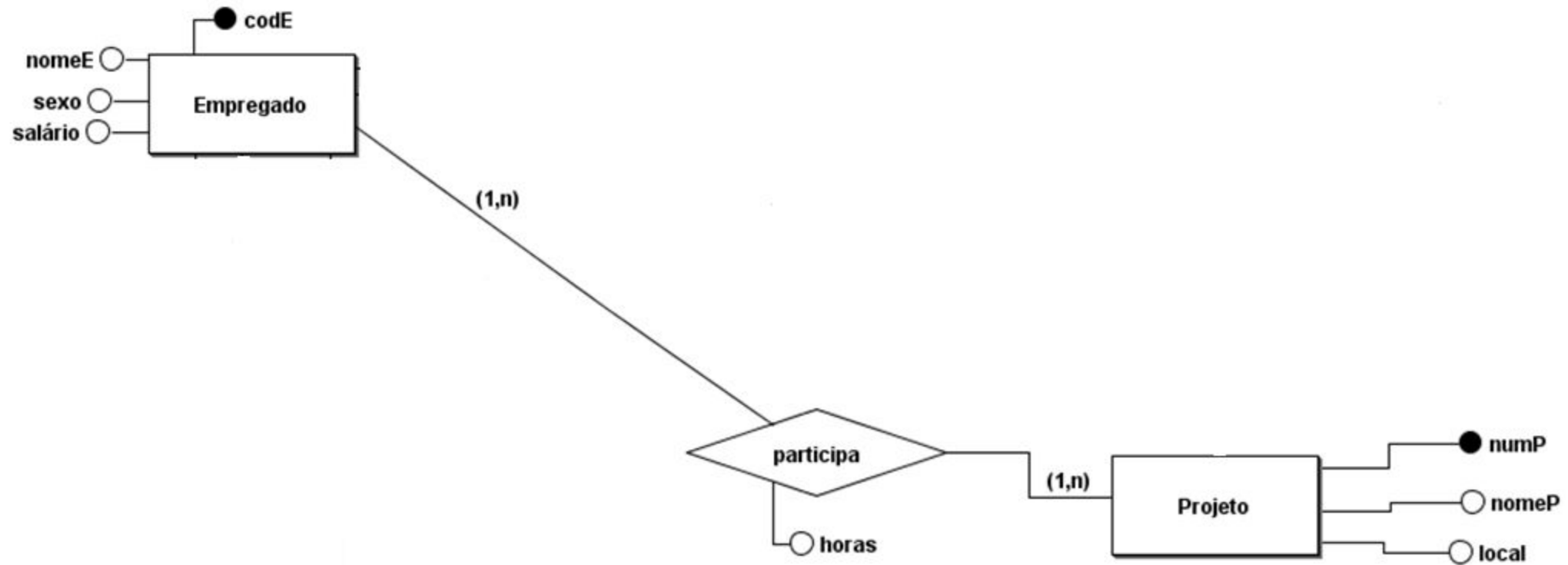
Passo 3 - Relacionamentos (1:N)



Empregado (codE, nomeE, sexo, salário, $codS_{FK}$, $numD_{FK}$)

DER □ Modelo Relacional

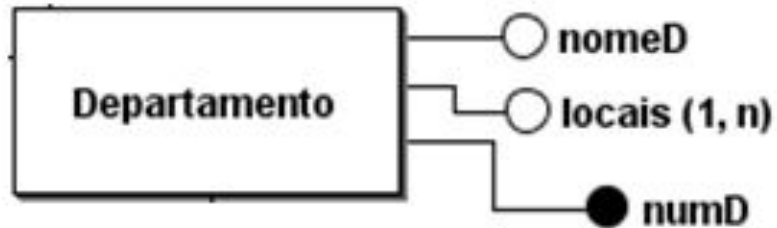
Passo 3 - Relacionamentos (M:N)



Empregado_Participa_Projeto (*codE*_{FK}, *numP*_{FK}, horas)

DER □ Modelo Relacional

Passo 4 - Atributos Multivalorados



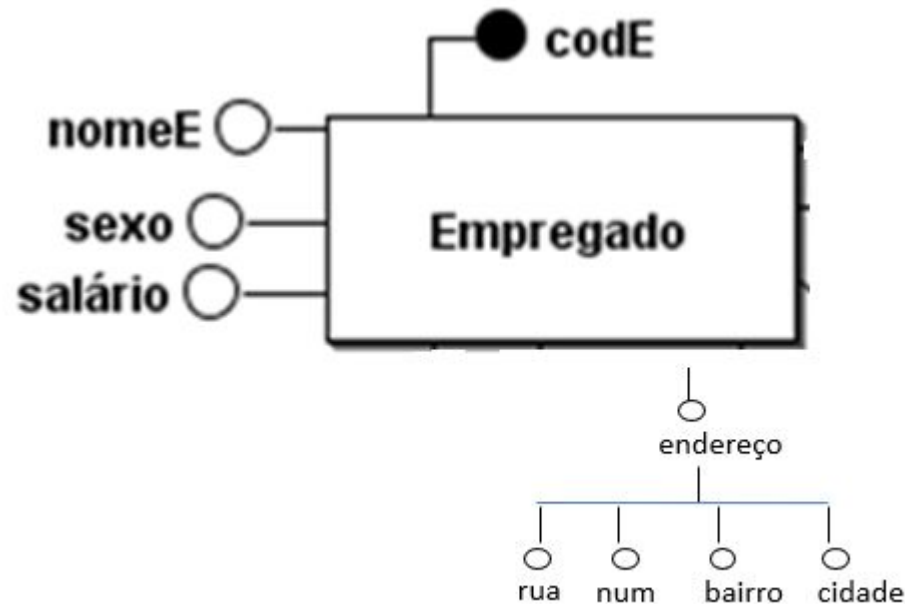
Alternativa 1: Local (numD_{FK}, nLocal)

Alternativa 2: Departamento (numD, nomeD, local1, local2)

PS: A alternativa 2 só se aplica se o número máximo de variações é conhecido.

DER □ Modelo Relacional

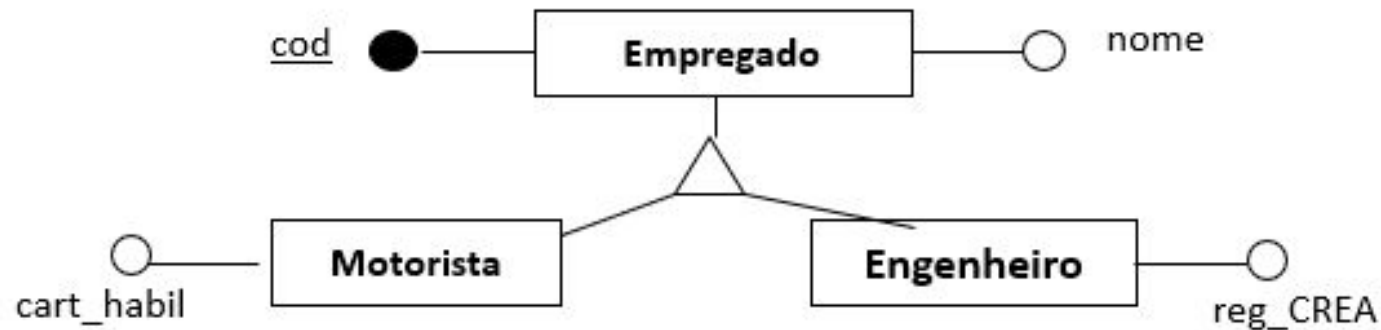
Passo 5 - Atributos Compostos



Empregado (codE, nomeE, sexo, salário, rua, num, bairro, cidade) ou
Endereço (codEnd, rua, num, bairro, cidade, codE_{FK})

DER □ Modelo Relacional

Passo 6 - Generalização/Especialização



Alternativa 1: uma tabela para cada, com dados comuns em uma delas e sem redundância.

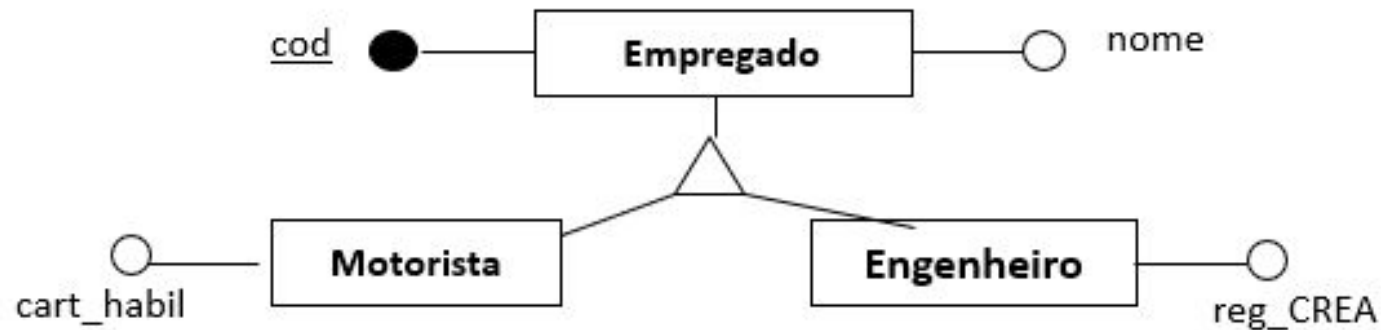
Empregado (cod, nome)

Motorista (cod (FK), cart_habil)

Engenheiro (cod (FK), reg_CREA)

DER □ Modelo Relacional

Passo 6 - Generalização/Especialização



Alternativa 2: uma tabela para cada, com redundância.

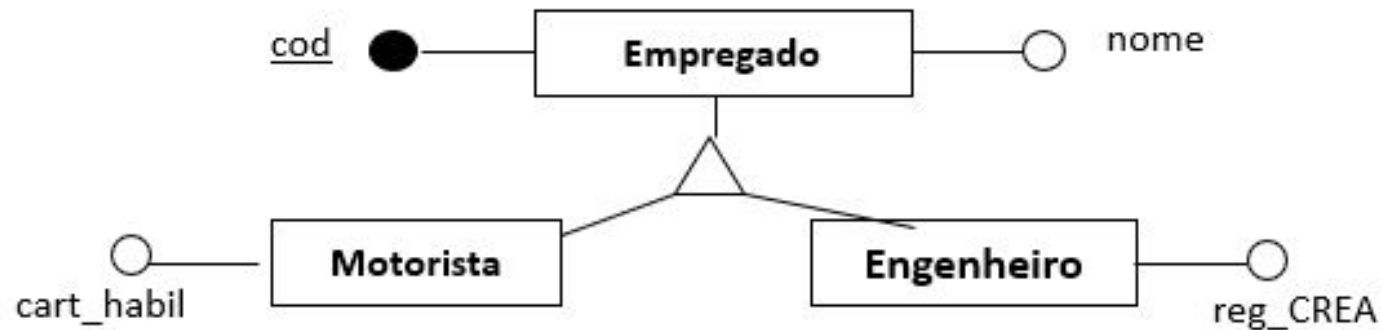
Empregado (cod, nome)

Motorista (cod (FK), nome, cart_habil)

Engenheiro (cod(FK), nome, reg_CREA)

DER □ Modelo Relacional

Passo 6 - Generalização/Especialização



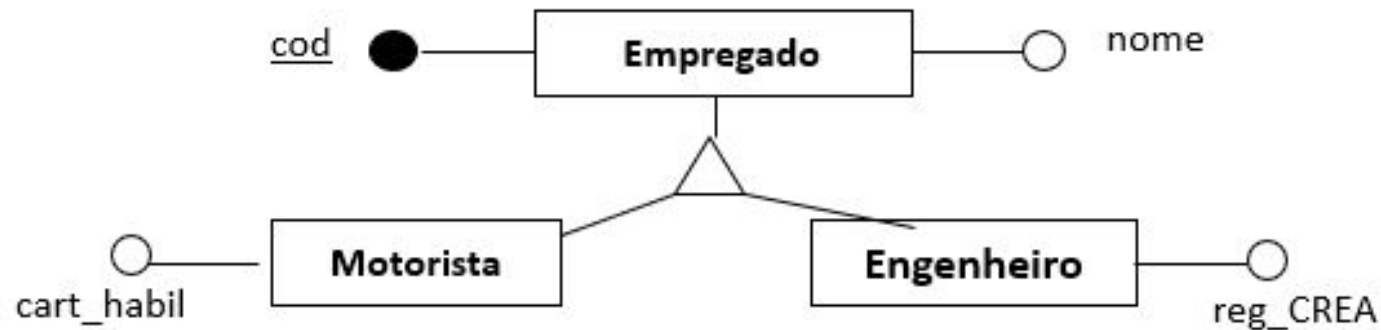
Alternativa 3: se mantém apenas as especializações.

Motorista (cod, nome, cart_habil)

Engenheiro (cod, nome, reg_CREA)

DER □ Modelo Relacional

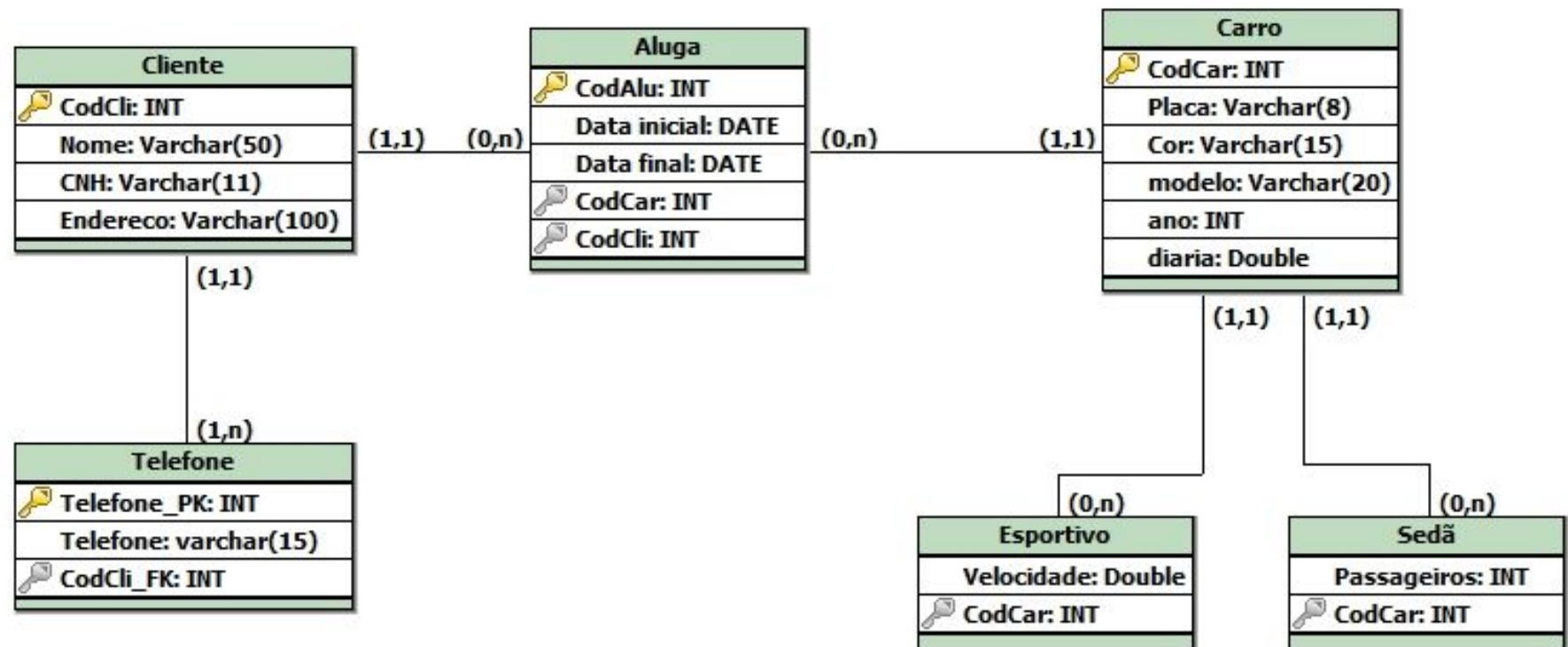
Passo 6 - Generalização/Especialização



Alternativa 4: se mantém apenas a tabela geral (tabela única).
Empregado (cod, nome, cart_habil, reg_CREA)

Modelo Lógico

- Modelo Lógico é a representação gráfica de um Modelo Relacional.
- Quando implementamos esse modelo, temos o que chamamos de Modelo Físico de Bancos de Dados.



Principais Tipos de Dados (MySQL)

- *Dados numéricos:*
 - TINYINT: número inteiro muito pequeno;
 - SMALLINT: número inteiro pequeno;
 - MEDIUMINT: número inteiro de tamanho médio;
 - **INT: número inteiro de tamanho comum;**
 - BIGINT: número inteiro de tamanho grande;
 - **DECIMAL: número decimal, de ponto fixo;**
 - FLOAT: número de ponto flutuante de precisão simples (32 bits)
 - DOUBLE: número de ponto flutuante de precisão dupla (64 bits)
 - BOOLEAN: quando 0, é considerado falso, quando 1 (ou outro valor diferente de 0), é considerado verdadeiro.
 - BIT: campo de um bit

Principais Tipos de Dados (MySQL)

- *Dados de Strings:*
 - CHAR: uma cadeia de caracteres, de tamanho fixo e não-binária;
 - **VARCHAR: uma *string* de tamanho variável e não-binária;**
 - BINARY: uma *string* binária de tamanho fixo;
 - VARBINARY: uma *string* binária de tamanho variável;
 - **BLOB: Objeto Grande Binário (*Binary Large Object*) de tamanho pequeno;**
 - TINYBLOB: um BLOB muito pequeno;
 - MEDIUMBLOB: um BLOB de tamanho médio;
 - LONGBLOB: um BLOB grande;
 - TINYTEXT: uma *string* não-binária e de tamanho bem reduzido;
 - TEXT: uma *string* não-binária e pequena;
 - MEDIUMTEXT: uma *string* de tamanho comum e não-binária;
 - LONGTEXT: uma *string* não-binária de tamanho grande;
 - ENUM: é uma string, com um valor que precisa ser selecionado de uma lista predefinida na criação da tabela;
 - SET: é um objeto que pode ter zero ou mais valores – cada um dos quais precisa ser escolhido de uma lista de valores predeterminados quando da criação da tabela.

Principais Tipos de Dados (MySQL)

- *Dados de Data e Tempo:*

- DATE: o valor referente a uma data no formato 'CCYY-MM-DD' (ano-mês-dia). O 'CC' se refere aos dois dígitos do século (em inglês, *Century*);
- TIME: um valor horário no formato 'hh:mm:ss' (horas:minutos:segundos);
- **DATETIME: uma combinação de data e hora. No MySQL, os dados deste tipo são exibidos no formato 'CCYY-MM-DD hh:mm:ss';**
- TIMESTAMP: *timestamp* é uma sequência de caracteres ou informação codificada que identifica uma marca temporal ou um dado momento em que um evento ocorreu. No MySQL, ele tem o formato 'CCYY-MM-DD hh:mm:ss';
- YEAR: armazena um ano no formato 'CCYY' ou 'YY'.

Anomalias

- Anomalias são mudanças em dados que podem gerar uma inconsistência no banco de dados relacional.
- Uma inconsistência é geralmente representada por situações em que dados que deveriam ser iguais, apresentam valores diferentes em várias tabelas do banco de dados.
- Por exemplo, o valor de compra de um produto deve ser o mesmo valor armazenado nas tabelas recibo e nota fiscal.
- Desse modo, se os valores forem diferentes, entende-se geralmente que existe uma inconsistência.
- Tal inconsistência geralmente aparece quando o banco de dados é projetado de forma inadequada.

Anomalias

AgenciaFuncionario	
FuncN	INT
Nome	VARCHAR(100)
Cargo	VARCHAR(45)
Salario	DECIMAL(2)
NumAg	INT
Endereco	VARCHAR(250)
Telefone	VARCHAR(250)
Indexes	

FuncN	Nome	Cargo	Salário	NumAg	Endereço	Tel
25	Luiz	Caixa	2000	1632	Prudente de Moraes, 15	3605-5223
30	Ricardo	Gerente	5000	1668	Hermes da Fonseca, 20	3605-5223
31	Josita	Caixa	2000	1632	Prudente de Moraes, 15	4225-5889
32	Francisca	Gerente	5600	1632	Prudente de Moraes, 15	4225-5889
33	Andréia	Caixa	2300	1668	Hermes da Fonseca, 20	5223-8556

Anomalias

- Assim, na imagem vista anteriormente, temos uma tabela chamada AgenciaFuncionario, que armazena os dados dos funcionários e de agências bancárias.
- Como essas duas informações (funcionário e agência) são armazenadas na mesma tabela, é possível também saber onde cada funcionário trabalha (agência, endereço, telefone).
- Ou seja, à primeira vista, a tabela AgenciaFuncionario parece ser uma ótima opção para reduzir o número de tabelas e aumentar a velocidade do sistema, entretanto, tal solução pode gerar várias anomalias.

Anomalias de Inserção

- Uma **anomalia de inserção** acontece quando, ao inserir um dado, este pode gerar uma inconsistência no banco de dados.
- No exemplo da imagem, para inserir os detalhes (NumAg, Endereço, Tel.) de um novo funcionário, você deve tomar cuidado para usar exatamente os dados já cadastrados por outros funcionários.
- Por exemplo, um novo funcionário para a agência 1668 deve usar exatamente o mesmo endereço dos outros dois funcionários que também trabalham nessa agência.
- Se isso não for feito, teremos um problema de inconsistência de dados, no qual dois funcionários que trabalham na mesma agência possuem endereços diferentes.

Anomalias de Remoção

- Uma **anomalia de remoção** acontece quando, ao remover um registro, você pode gerar inconsistência no banco de dados.
- No exemplo da imagem, uma anomalia de remoção acontece quando, por exemplo, removemos o funcionário de número 30.
- Nesse caso, o objetivo é apenas remover os dados do funcionário e preservar os dados da agência 1668. Entretanto, da forma como a tabela está estruturada, os dados da agência também são removidos.

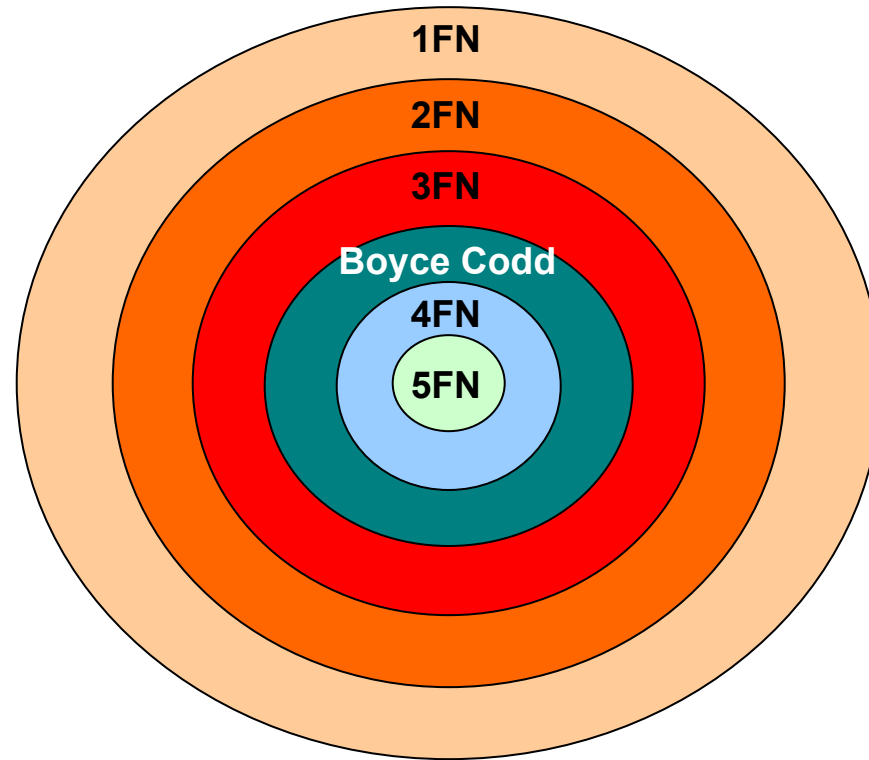
Anomalias de Atualização

- Uma **anomalia de atualização** acontece quando, ao atualizar um registro, você pode gerar inconsistência no banco de dados.
- No exemplo da imagem, uma anomalia de atualização acontece quando, por exemplo, modificamos o endereço da agência 1632.
- Nesse caso, teremos que atualizar o endereço de todos os funcionários da agência 1632. Caso contrário, teremos uma inconsistência no banco de dados no qual funcionários da mesma agência possuem endereços diferentes.
- Note que todas as anomalias acontecem devido à existência de redundância de informação na tabela AgenciaFuncionario. Por exemplo, o mesmo endereço de uma agência é armazenado várias vezes quando ele poderia ser armazenado apenas uma vez.
- Assim, para evitar as anomalias é preciso evitar a redundância. A redundância é evitada através da **normalização das tabelas**.

Normalização

- Minimizar redundâncias
- Eliminar anomalias de atualização
- Prover o melhor caminho de acesso
- Assegurar consistência na manutenção de modelo de dados
- Evitar dados não identificáveis através de definição rigorosa de identificadores e relacionamentos

Normalização



Normalização

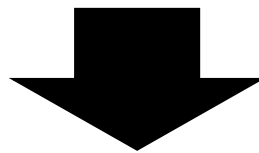
PROJETO (CodProj, Tipo, Descricao, (CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado))

Exemplo tem uma tabela embutida (atributo composto)

CodProj	Tipo	Descricao	Empregado					
			CodEmpregado	Nome	Categoria	Salario	DataInicio	TempoAlocado
PJ554	Desenvolvimento	Sistema de estoque	1	João	A1	6	01/01/2019	12
			2	Maria	A1	6	01/01/2019	12
			7	José	B2	4	01/01/2019	12
			8	Angelo	B3	2	30/03/2019	9
			9	Márcia	A2	5	15/04/2019	6
			10	Pedro	A2	5	15/04/2019	6
PJ529	Desenvolvimento	Sistema de compra	1	João	A1	6	01/01/2020	8
			2	Maria	A1	6	01/01/2020	3
			7	José	B2	4	01/01/2020	6
			8	Angelo	B3	2	01/01/2020	5
			9	Márcia	A2	5	15/11/2019	5
			10	Pedro	A2	5	15/11/2019	8
PF38	Correção	Problema no upload	3	Augusto	A1	6	01/01/2018	3
			4	Joana	B2	4	01/01/2018	1
			5	Alex	B3	2	01/01/2018	5
			6	Sophie	B3	2	01/01/2018	2
			7	José	A2	4	01/01/2020	6
			8	Angelo	B3	2	01/01/2020	5

Normalização - 1FN (Primeira Forma Normal)

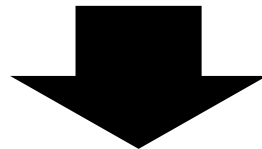
- Os atributos que compõem uma tabela devem ser todos atômicos (não podem ser divisíveis).
- Não devem conter outras tabelas embutidas.
- **PROJETO** (CodProj, Tipo, Descricao, (CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado))



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado).

Normalização - 2FN (Segunda Forma Normal)

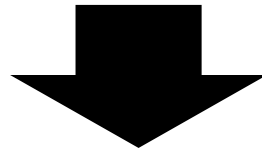
- Deve estar na 1FN e não possuir **dependências parciais**.
- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, Nome, Categoria, Salario, DataInicio, TempoAlocado).



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Salario).

Normalização - 3FN (Terceira Forma Normal)

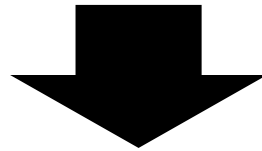
- Deve estar na 2FN e não possuir **dependências transitivas**.
- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Salario).



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **CATEGORIA** (Categoria, Salario).

Normalização - Boyce Codd

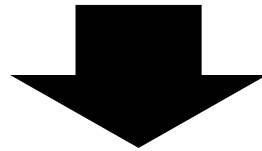
- Os atributos de todas as tabelas devem depender única e exclusivamente de suas chaves-primárias.
- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **CATEGORIA** (Categoria, Salario).



- **PROJETO** (CodProj, Tipo, Descricao);
- **ALOCACAO_EMPREGADO** (CodProj, CodEmpregado, DataInicio, TempoAlocado);
- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **CATEGORIA** (Categoria, Salario).

Normalização - 4FN (Quarta Forma Normal)

- Deve estar na 3FN e não possuir **dependências multivaloradas**.
- **EMPREGADO** (CodEmpregado, Nome, Categoria, Fones(1,n));



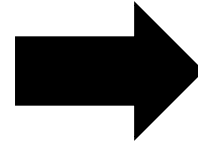
- **EMPREGADO** (CodEmpregado, Nome, Categoria);
- **FONES** (CodEmpregado, NFone).

Normalização - 5FN (Quinta Forma Normal)

- Deve estar na 4FN e não possuir **constantes simétricas**.
- Normalmente, tabelas que estão na 4FN já se encontram na 5FN.

Normalização - 5FN (Quinta Forma Normal)

[PK][FK] seller	[PK][FK] company	[PK][FK] product
1	Samsung	Hard drives
1	Samsung	Phones
1	Apple	Hard drives
1	Apple	Phones



[PK][FK] seller	[PK][FK] company
1	Samsung
1	Apple

[PK][FK] seller	[PK][FK] product
1	Hard drives
1	Phones

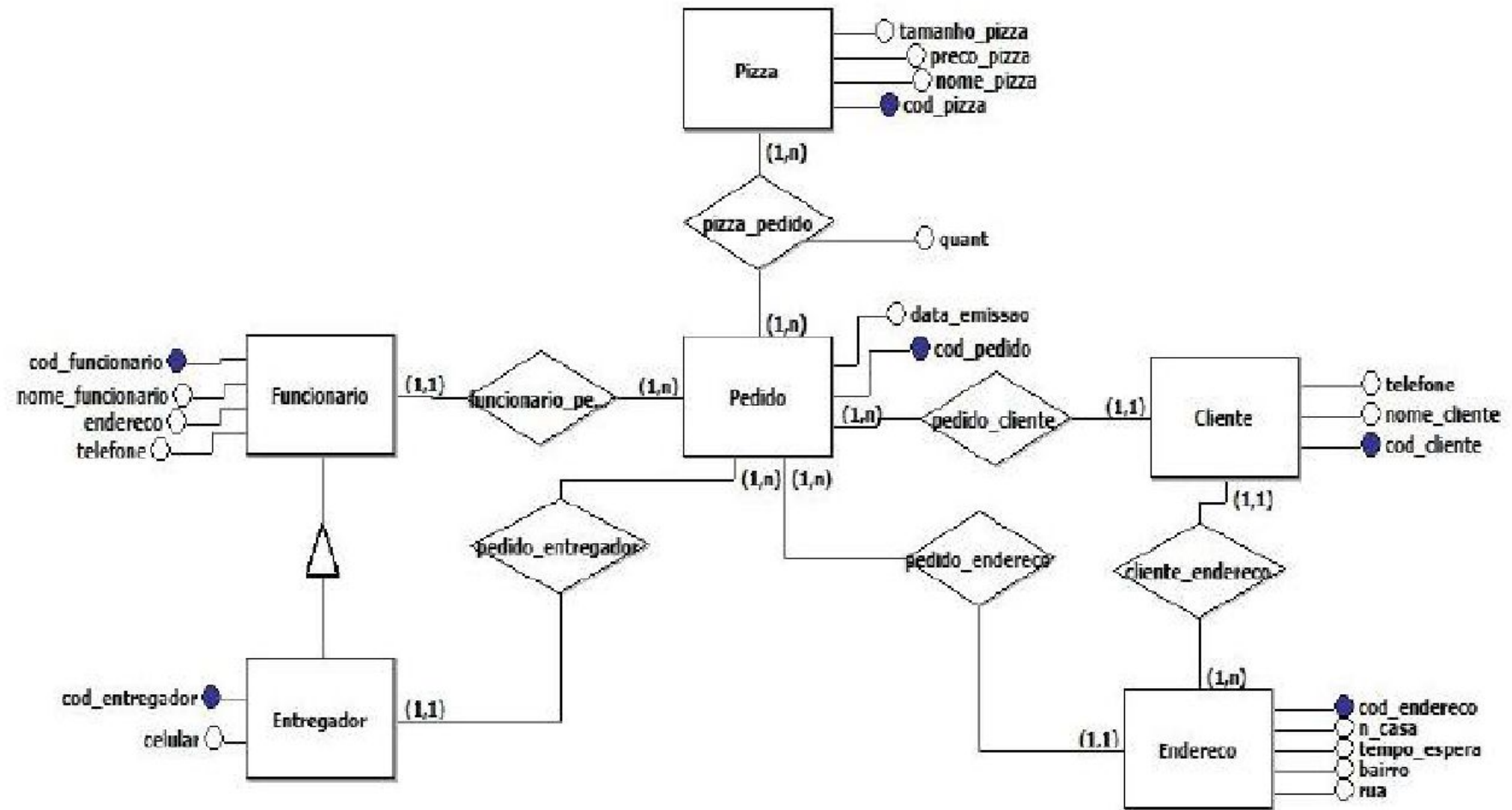
[PK][FK] company	[PK][FK] product
Samsung	Hard drives
Samsung	Phones
Apple	Hard drives
Apple	Phones

Normalização - Resumo

- Eliminar atributos não atômicos □ 1FN
- 1FN + Eliminar dependências funcionais parciais □ 2FN
- 2FN + Eliminar dependências transitivas □ 3FN
- 3FN + Eliminar dependências multivaloradas □ 4FN
- 4FN + Garantir ausência de constantes simétricas □ 5FN

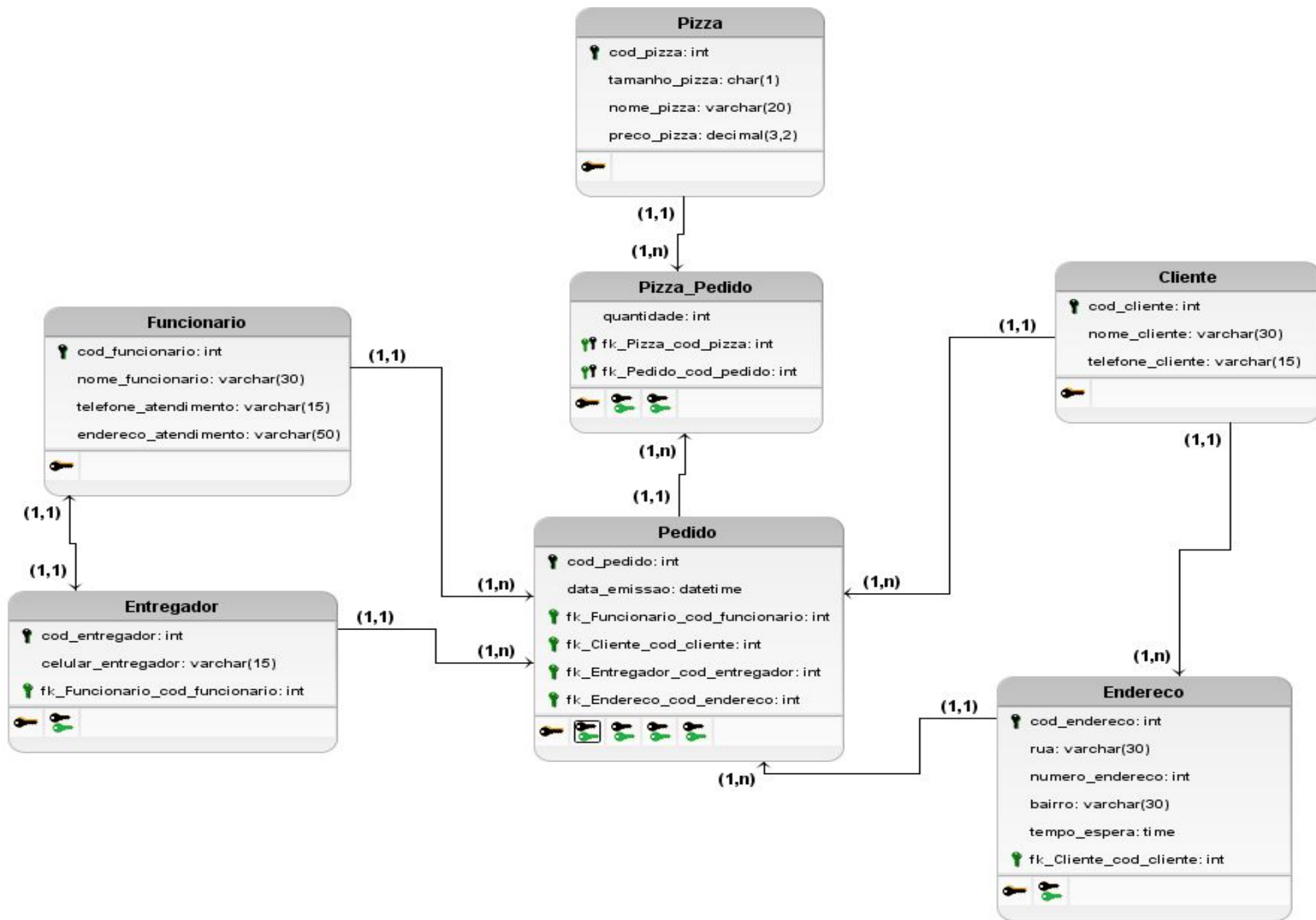
- Garantir dependências dos atributos apenas com suas chaves-primárias □ FN de Boyce Codd.

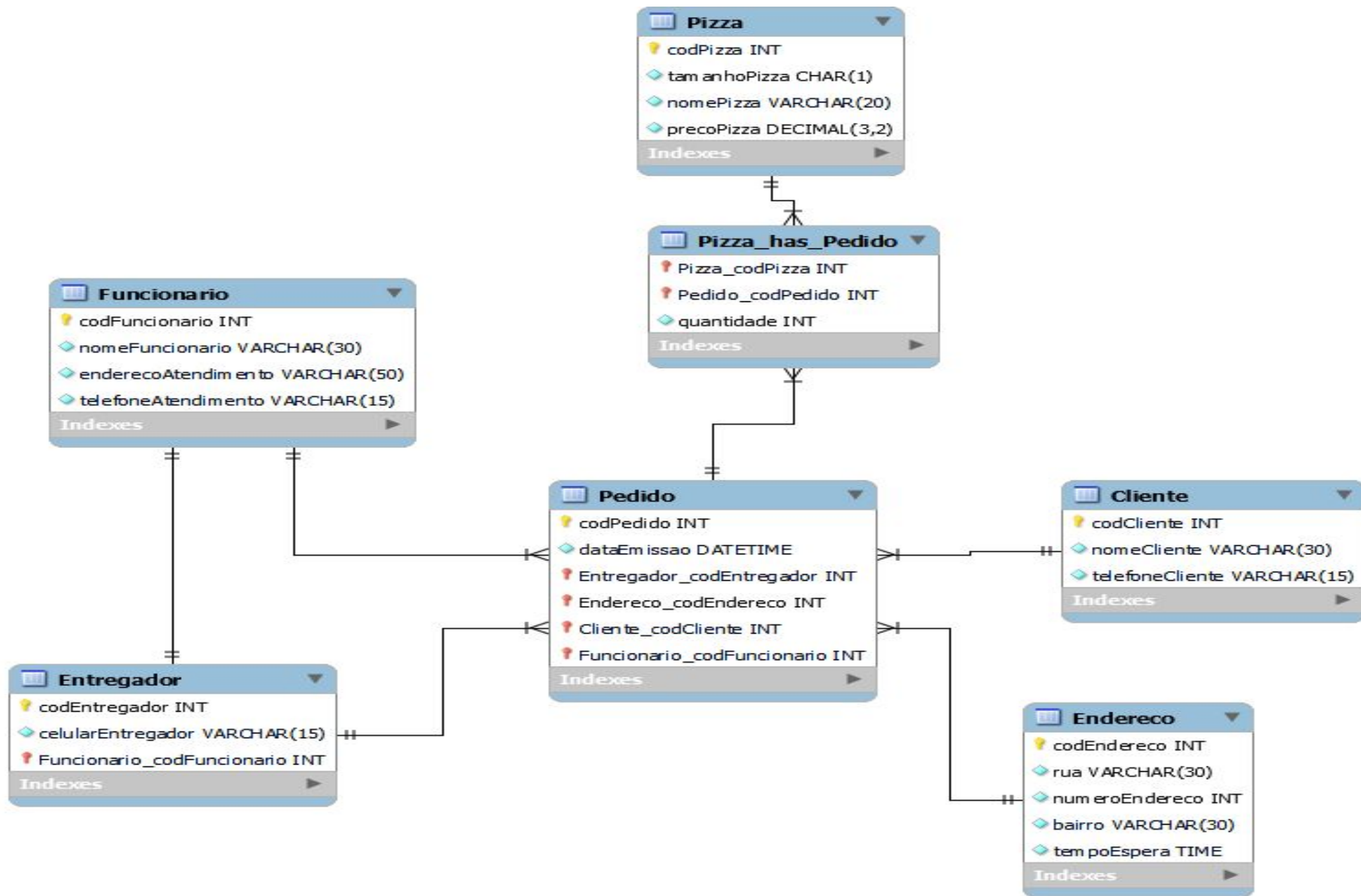
Exercícios - 1ª Questão



Exercícios - 1ª Questão

Transforme o modelo conceitual do sistema de controle de pizzeria em um modelo relacional.





Referências

- COUGO, Paulo. Modelagem conceitual e projeto de banco de dados. São Paulo: Elsevier, 2017.
- MACHADO, Felipe Nery Rodrigues; ABREU, Maurício Pereira de. Projeto de banco de dados: uma visão prática. 17. ed. São Paulo, SP: Érica, 2013.
- RAMAKRISHNAN, Raghu; GEHRKE, Johannes. Sistemas de gerenciamento de banco de dados. São Paulo, SP: McGraw-Hill, c2008.

COMPLEMENTAR

- DATE, C. J. Introdução a sistemas de bancos de dados. [8. ed.], 7. tiragem. Rio de Janeiro, RJ: Elsevier, c2004.
- DATE, C. J. Projeto de banco de dados e teoria relacional. São Paulo: Novatec, 2015.
- ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 6. ed. 3. reimp. São Paulo, SP: Pearson Addison Wesley, 2014.
- OPPEL, Andy; SHELDON, Robert. SQL: um guia para iniciantes. 3. ed. Rio de Janeiro, RJ: Ciência Moderna, 2009.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de banco de dados. 6 ed. São Paulo, SP: Makron books, 2012.

Modelo Lógico + Normalização

Prof. Cloves Rocha

Ciência da Computação - CC